CS 61A Midterm 2 Study Guide

Values that are false in a boolean context:

- `0` and `0.0`
- `False` and `None`
- An empty container, such as `''`, `[]`, `()`, `{}`

Others are true values.

```
>>> bool(0)
False
>>> bool('0')
True
>>> bool([[]])
True
>>> bool(lambda x: 0)
True
```

Membership

```
>>> digits = [1, 8, 2, 8]
>>> 2 in digits
True
>>> 1828 not in digits
True
>>> [1, 8] in digits
False
>>> 'd' in 'hotdog'
True
>>> 'dog' in 'hotdog'
True
```

Error Types

IndexError	indexing a list past its last element
KeyError	a key isn't in a dictionary
TypeError	an operation on the wrong type: calling a function with wrong number of arguments; indexing or calling None
AttributeError	an attribute is not found for an object
NameError	A name is not bound in the environment

Lambda & Currying: e.g. `f(x, y) → f(x)(y)`

```
>>> square = lambda x: x * x   # anonymous function
>>> curry2 = lambda f: lambda x: lambda y: f(x, y)
>>> square(4), curry2(add)(3)(4)
(16, 7)
```

Function Inputs, Outputs, & Type Hints

Always ask: **what type goes in, what type comes out?**

```
def sum_list(s: list[int]) -> int:
    ...
```

Python **doesn't check or enforce** type hints.
They document a function's intended behavior.

list[int]	list of ints	None	no return
dict[str,int]	str → int	A | B	an A or a B
tuple[int,str]	(int,str) pair	tuple[()]	()
Link[int]	T = int	LinkedList	Link or ()

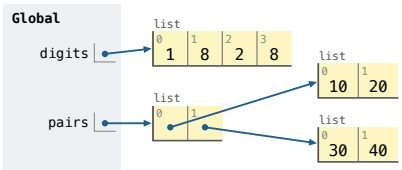
Type variables: A name, usually T, is used to indicate that the same type is involved in two or more formal parameters or return values.

```
def pair[T](a: T, b: T) -> list[T]:
    return [a, b]
```

Lists

```
>>> digits = [1, 8, 2, 8]
>>> len(digits)
4
>>> digits[3]
8
>>> digits[-1]  # last
8
>>> [2, 7] + digits * 2
[2, 7, 1, 8, 2, 8, 1, 8, 2, 8]
```

```
>>> pairs = [[10, 20],
...          [30, 40]]
>>> pairs[1]
[30, 40]
>>> pairs[1][0]
30
```



Slicing: `s[<start>:<end>]` makes a new list: from `<start>` (**included**) up to `<end>` (**not included**).
Defaults: start=0, end=len(s)
`s[:]` and `list(s)` **copy** the whole list.

```
>>> s = [6, 5, 4, 3, 2, 1, 0]
>>> s[1:3]
[5, 4]
>>> s[:3]
[6, 5, 4]
>>> s[3:]
[3, 2, 1, 0]
>>> s[-3:]  #last three elements
[2, 1, 0]
```

```
>>> s[:]
[6, 5, 4, 3, 2, 1, 0]
>>> s[10:]
[]
>>> empty = []
>>> empty[0]
IndexError
>>> empty[1:]
[]
```

A slice of a list always evaluates to a list (maybe empty) and never causes an IndexError.

Iteration: for Statements

```
for <name> in <expression>:
    <suite>
```

1. Evaluate the header `<expression>`, which must be an **iterable** value (a list, tuple, range, dict, etc.)
2. For each element in that iterable value, in order:
   - Bind `<name>` to that element in the current frame
   - Execute the `<suite>`

Unpacking in a for statement:

```
>>> pairs = [[1, 2], [2, 2],
...          [3, 2], [4, 4]]
>>> same_count = 0
>>> for x, y in pairs:
...     if x == y:
...         same_count += 1
>>> same_count
2
```

pairs is a sequence of fixed-length sequences

A name for each element in a fixed-length sequence

Ranges

`range(-2, 2)` contains ..., −3, **−2, −1, 0, 1**, 2, 3, ...

- **Length:** ending value − starting value.
- **Element selection:** starting value + index.

```
>>> r = range(-2, 2)
>>> r
range(-2, 2)
>>> len(r)
4
```

```
>>> list(range(-2, 2))
[-2, -1, 0, 1]
>>> list(range(4))
[0, 1, 2, 3]
```

Aggregation Functions for an iterable s

sum(s, start=0)	*total*
max(s, key=None), **max**(a, b, ..., key=None)	*largest*
min(s, key=None), **min**(a, b, ..., key=None)	*smallest*
all(s)	*all true?*
any(s)	*any true?*
len(s)	*length*

```
>>> all([False, True])
False
>>> all([])
True
>>> sum([1, 2])
3
>>> sum([1, 2], 3)
6
>>> sum([])
0
>>> sum([[1], [2]], [])
[1, 2]
```

```
>>> any([False, True])
True
>>> any([])
False
>>> max(1, 2)
2
>>> max([1, 2])
2
>>> max([1, -2], key=abs)
-2
>>> len([[1, 2], 3])
2
```

Dictionaries

A container of key–value pairs; keys must be unique; iteration gives the **keys**; d[k] errors on a missing key; d.get(k, default) returns default if k is not a key.

```
>>> words = {"más": "more",
...          "otro": "other",
...          "agua": "water"}
>>> len(words)
3
>>> "agua" in words
True
>>> "water" in words
False
>>> "water" in words.values()
True
```

```
>>> words["otro"]
'other'
>>> words["hola"]
KeyError
>>> words.get("otro", "hi")
'other'
>>> words.get("hola", "hi")
'hi'
>>> list(words)
['más', 'otro', 'agua']
>>> list(words.values())
['more', 'other', 'water']
```

List & Dictionary Comprehensions

`[<map exp> for <name> in <iter exp> if <filter exp>]`

Short version: `[<map exp> for <name> in <iter exp>]`

A combined expression that evaluates to a list by:
1. For each item in the iterable value of `<iter exp>`, bind `<name>` to that item, then if `<filter exp>` evaluates to a true value, include the value of `<map exp>` in the resulting list
2. `<name>` is not bound in the current frame.

```
>>> [word for word in words if len(word) == 4]
['otro', 'agua']
>>> [word for word in words]
['más', 'otro', 'agua']
>>> [words[word] for word in words]
['more', 'other', 'water']
>>> {x: x**x for x in range(3,6)}  # dict comprehension
{3: 9, 4: 16, 5: 25}
>>> x
NameError
```

Tuples

```
>>> empty = ()
>>> len(empty)
0
>>> print(empty)
()
>>> (2, 3) + (4, 5)
(2, 3, 4, 5)
```

```
>>> conditions = ('rain',
'shine')
>>> conditions[0]
'rain'
>>> ra, sh = conditions
>>> sh
'shine'
```

Classes with @dataclass

A **class** statement creates a new type. When decorated with **@dataclass**, each name that has a type hint becomes an attribute of the instances of this new class. The `__str__` method within the class determines the output when str or print is called on an instance.

```
@dataclass
class Position:
    "A geographic position."
    lat: float
    lon: float

    def __str__(self):
        return format_pos(self)

def format_pos(p: Position) -> str:
    "Format the position with directional suffixes."
    assert abs(p.lat) <= 90 and abs(p.lon) <= 180, \
        f'{p!r} is not on Earth!'
    ... # formats a Position as 33.87° S, 151.21° E
```

```
>>> sydney = Position(-33.87, 151.21)
>>> sydney.lat
-33.87
>>> type(sydney) == Position
True
>>> print(sydney)
33.87° S, 151.21° E
```

Recursion

State what the function **takes in** and **should return.**
sum_digits(n): takes a positive int n; returns the sum of its digits.

- **Base case:** an input simple/small enough that the return value can be found without recursion. One digit: return n.
- **Recursive case:** call on a **simpler/smaller input** and rely on the recursive call's correctness: sum_digits(all_but_last) returns the digit sum of all but the last digit; add last.

```
def sum_digits(n: int) -> int:
    "Sum the digits of positive integer n."
    if n < 10:
        return n
    else:
        all but last, last = n // 10, n % 10
        return sum_digits(all_but_last) + last
```

- **def header:** like any function
- Conditionals: **base cases**
- Base: **no recursive calls**
- Recursive: **recursive calls**

Tree Recursion

A recursive case makes **more than one** recursive call.

```
def fib(n: int) -> int:
    if n == 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fib(n-2) + fib(n-1)
```

n	fib(n)
0	0
1	1
2	1
3	2
4	3

n	fib(n)
5	5
6	8
7	13
8	21

Recursive decomposition: find simpler instances of the same problem that complete most of the work.
count_partitions(n, m) = ways to write n as a sum of parts up to size m. For (6, 4), explore two possibilities:

- **use at least one 4:** what's left is a partition of 2 → count_partitions(2, 4)
- **don't use any 4:** parts up to 3 → count_partitions(6, 3)

The last step of the recursive case is to add the two counts to get a total count for (6, 4).

```
def count_partitions(n: int, m: int) -> int:
    if n == 0:
        return 1
    elif n < 0:
        return 0
    elif m == 0:
        return 0
    else:
        with_m = count_partitions(n-m, m)
        without_m = count_partitions(n, m-1)
        return with_m + without_m
```

Linked Lists

A linked list is either **empty** () or a **Link** with a **first** value and the **rest** of the list.

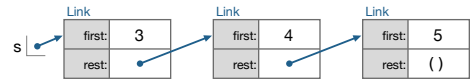
- An empty list is (). Test if s is empty with `not isinstance(s, Link)` or `s == ()`.
- There's no indexing: to reach item k, follow .rest k times.

```
type LinkedList[T] = Link[T] | tuple[()]

@dataclass
class Link[T]:
    first: T
    rest: LinkedList[T] = ()  # empty list

    def __str__(self):
        return format_link(self)

def format_link(s: Link) -> str:
    """Return a Link s formatted as items within parentheses."""
    string = '(' + str(s.first)
    remaining = s.rest
    while isinstance(remaining, Link):
        string += ' ' + str(remaining.first)
        remaining = remaining.rest
    assert remaining == (), f'{s!r} is not a LinkedList'
    return string + ')'
```




```
>>> end = Link(5)
>>> s = Link(3, Link(4, end))
>>> s.first
3
>>> s.rest.first
4
>>> s.rest.rest.first
5
>>> s.rest.rest.rest == ()
True
>>> isinstance((), Link)
False
```

```
>>> isinstance(s, Link)
True
>>> s.rest.rest
Link(first=5, rest=())
>>> s.rest.rest == end
True
>>> print(s)
(3 4 5)
>>> print(end)
(5)
>>> print(Link(end, s))
((5) 3 4 5)
```

Examples

```
def len_link(s: LinkedList) -> int:
    """Return the length of a linked list.

    >>> len_link(Link(3, Link(4, Link(5))))
    3
    """
    length = 0
    while isinstance(s, Link):
        length += 1
        s = s.rest
    return length
```

Stops when s is ()

```
def len_link(s: LinkedList) -> int:
    if not isinstance(s, Link):
        return 0
    return 1 + len_link(s.rest)
```

Base case: s is ()

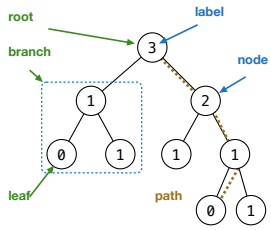
```
def range_link(start: int, end: int) -> LinkedList[int]:
    if start >= end:
        return ()
    else:
        return Link(start, range_link(start + 1, end))
```

CS 61A Midterm 3 Study Guide

Trees

Recursive description:
- A **tree** has a root **label** and a list of **branches**
- Each branch is a **tree**
- A tree with zero branches is a **leaf**: not t.branches

Relative description:
- Each location is a **node**
- Each **node** has a **label**
- Branch roots are **children** of their **parent**
- A **path** runs from the root down to a node




```
from __future__ import annotations
from dataclasses import dataclass, field

@dataclass
class Tree[T]:
    """A Tree has a label (of type T) and a list of branches."""
    label: T
    branches: list[Tree[T]] = field(default_factory=list)  # []

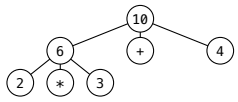
    def __str__(self):
        return format_tree(self)

def format_tree(t: Tree, indent='') -> str:
    "Format a tree with each branch indented below its label."
    assert isinstance(t, Tree), f'{t!r} is not a Tree'
    assert isinstance(t.branches, list), f'branches of {t!r} is
not a list'
    string = indent + str(t.label)
    for b in t.branches:
        string += '\n' + format_tree(b, indent + '  ')
    return string

def is_leaf(t: Tree) -> bool:
    "Return whether a Tree t is a leaf with no branches."
    return not t.branches
```

```
>>> expr = Tree(10, [Tree(6, [Tree(2), Tree('*'),
...   Tree(3)]), Tree('+'), Tree(4)])
>>> expr.branches[0].label, len(expr.branches)
(6, 3)
```

```
>>> print(expr)
10
  6
    2
    *
    3
  +
  4
```

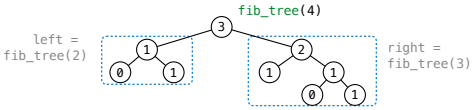



```
>>> expr.branches[1]
Tree(label='+', branches=[])
```

Tree-Recursive Functions

Each branch is a **smaller tree**, so a call on a branch keeps the same promise. Recurse on every branch, then combine.

```
def fib_tree(n: int) -> Tree:
    if n == 0 or n == 1:
        return Tree(n)
    else:
        left = fib_tree(n-2)
        right = fib_tree(n-1)
        fib_n = left.label + right.label
        return Tree(fib_n, [left, right])
```



Mutation: List Methods

```
>>> suits = ['coin', 'string', 'myriad', 'cup']
>>> suits.pop()          # remove & return last
'cup'
>>> suits.pop(1)         # remove & return index 1
'string'
>>> suits.remove('myriad')  # first match
>>> suits.append('cup')     # add one
>>> suits.extend(['sword', 'club'])  # add all
>>> suits[2] = 'spade'
>>> suits[0:2] = ['diamond']  # replace slice
>>> suits.insert(0, 'heart')  # add at index
>>> suits
['heart', 'diamond', 'spade', 'club']
>>> print(suits.append('x'))  # mutating methods
None                          # return None
```

```
>>> s = [1, 2]
>>> s.append([3, 4])
>>> s.extend([5, 6])
>>> s
[1, 2, [3, 4], 5, 6]
```

```
>>> t = s
>>> s += [7]      # mutates
>>> s = s + [8]   # new list
>>> s
>>> t
[1, 2, [3, 4], 5, 6, 7]
```

Dictionaries are mutable too (keys must be immutable):

```
>>> words["oruguita"] = 'caterpillar'
>>> words["oruguita"] += '🍾'
>>> words["oruguita"]
'caterpillar🍾'
```

Mutation: Aliases & Copies

```
>>> a = [10]
>>> b = a      # same list
>>> a == b
True
>>> a.append(20)
>>> a == b
True
>>> a
[10, 20]
>>> b          # b too
[10, 20]
>>> a is b     # same object
True
```

```
>>> a = [10]
>>> b = [10]   # new list
>>> a == b
True
>>> b.append(20)
>>> a
[10]
>>> b
[10, 20]
>>> a == b
False
>>> a is b
False
```

A copy is shallow: nested lists are still shared.

```
>>> a = [[1], 2]
>>> c = list(a)
>>> c[0].append(5)
>>> c[1] = 9
>>> a
[[1, 5], 2]
```

A tuple can't change, but a list **inside** it can.

```
>>> c is a, c[0] is a[0]
(False, True)
>>> t = (1, [2])
>>> t[1].append(3)
>>> t
(1, [2, 3])
```

Iterators & Lazy Evaluation

An **iterable** is any value you can loop over (list, tuple, range, dict, str). An **iterator** steps through an iterable one element at a time and remembers its current **position.**

```
iter(iterable)  an iterator over the iterable's elements
next(iterator)  next element, or StopIteration if none
```

```
>>> s = [3, 4, 5]
>>> t = iter(s)
>>> next(t)
3
>>> next(t)
4
```

```
>>> d = {'one': 1,
...      'two': 2}
>>> k = iter(d)
>>> next(k)
'one'
>>> next(k)
'two'
```

```
>>> v = iter(
...   d.values())
>>> next(v)
1
>>> next(v)
2
```

An iterator is **one pass:** once it reaches the end it stays empty. Calling `iter` again gives a **new** one that starts over.

Lazy (nothing computed yet): map filter zip reversed

Eager (every element now): list tuple sorted sum max

```
map(func, iterable)     func(x) for x in iterable
filter(func, iterable)  x in iterable if func(x)
zip(first, second)      co-indexed (x, y) pairs
reversed(sequence)      x in reverse order
```

- To view the contents of an iterator, place the elements into a **container** with list, tuple or sorted.
- A **for loop** calls iter, then next until StopIteration.
- iter of an iterator returns **itself.**
- Adding a dict key while iterating over it: RuntimeError.
- **Lazy:** map(func, s) calls func on **one** element each time next asks; an eager function forces **all the rest** at once.

Generators

A **generator function** has **yield** in its body. Calling it **doesn't run the body:** it returns a **generator**, which is an iterator. Each next runs the body until the next **yield**, returns that value, and **pauses** there (local names keep their values). When the body ends, next raises StopIteration.

```
def countdown(n):
    print('start')
    while n > 0:
        yield n
        n -= 1
    print('done')
```

```
>>> g = countdown(2)
>>> next(g)
start
2
>>> next(g)
1
>>> next(g)
done
StopIteration
```

countdown(2) printed nothing; each next resumes after **yield.**

yield from <it> yields every value of <it>, one at a time: the same as for x in <it>: yield x.

```
def a_then_b(a, b):
    yield from a
    yield from b
```

```
>>> list(a_then_b([3, 4],
...          range(5, 7)))
[3, 4, 5, 6]
```

Recursive: trust the recursive call to yield everything for the smaller input.

```
def prefixes(s):
    if s:
        yield from prefixes(
            s[:-1])
        yield s
```

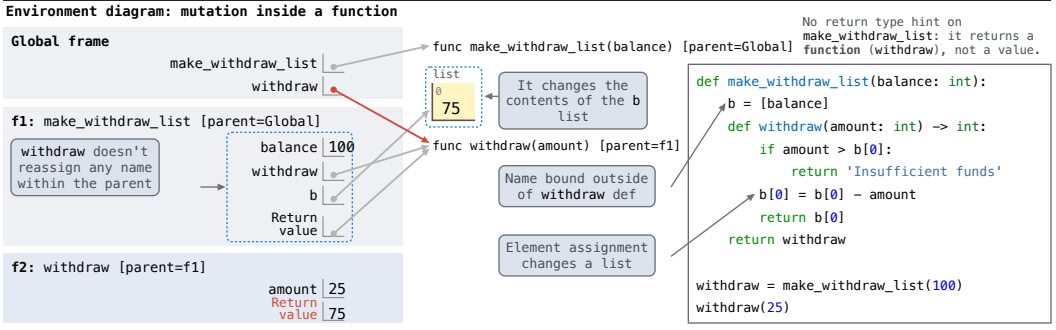
```
>>> list(prefixes('abc'))
['a', 'ab', 'abc']
```

Efficiency: Orders of Growth

Sublinear	grows slower than n: **constant** (indexing into a list), **logarithmic** (sum of digits of an int)
Linear	incrementing n adds a constant (one loop over s)
Superlinear	grows faster than n: **quadratic** (double for loop), **exponential** (tree recursion)

Memoization: save each result (e.g. in a dict keyed by the argument) so a repeated call is just a lookup instead of being recomputed. A memoized tree-recursive fib computes each fib(k) once: superlinear (in this case exponential) → **linear.**

Environment diagram: mutation inside a function



Python Object System

When a class is called:
1. A new instance of that class is created.
2. The __init__ method of the class is called with the new object as its first argument (named self), along with any additional arguments provided in the call expression.

Example: All bank accounts have a **balance** and an account **holder**; the **Account** class adds those attributes to each of its **instances.**

```
>>> a = Account('Jim')
>>> a.holder
'Jim'
>>> a.balance
0
```

A new instance is created by calling a class

An account instance
balance: 0 holder: 'Jim'

```
class Account:
    interest = 0.02   # class attribute: shared
    def __init__(self, account_holder):
        self.balance = 0    # instance attributes:
        self.holder = account_holder  # one per object
    def deposit(self, amount):
        self.balance = self.balance + amount
        return self.balance
    def withdraw(self, amount):
        if amount > self.balance:
            return 'Insufficient funds'
        self.balance = self.balance - amount
        return self.balance
```

__init__ is called a **constructor**

self should always be bound to an instance of the Account class or a subclass of Account

```
>>> type(Account.deposit)
<class 'function'>
>>> type(a.deposit)
<class 'method'>
```

Through the class: a plain **function.** Through an instance: a **bound method**

```
>>> Account.deposit(a, 5)
10
```

Function call: all arguments within parentheses

```
>>> a.deposit(2)
12
```

Method invocation: one object before the dot, other arguments within parentheses

Class vs. Instance Attributes

Assigning to <expression>.<name> sets an attribute on that object: on an **instance,** an instance attribute; on a **class,** a class attribute.

```
>>> jim = Account('Jim')
>>> tom = Account('Tom')
>>> tom.interest
0.02
>>> Account.interest = 0.04
>>> tom.interest
0.04
>>> jim.interest
0.04
```

```
>>> jim.interest = 0.08
>>> jim.interest
0.08
>>> tom.interest
0.04
>>> Account.interest = 0.05
>>> tom.interest
0.05
>>> jim.interest
0.08
```

Account (class)	jim (instance)	tom (instance)
interest: ~~0.02~~ ~~0.04~~ 0.05	balance: 0 interest: 0.08	balance: 0

Inheritance

A **subclass** inherits all attributes of its **base class** and can **override** some. Use it for **is-a** relationships (a checking account *is an* account).

```
class CheckingAccount(Account):
    """A bank account that charges for withdrawals."""
    withdraw_fee = 1
    interest = 0.01
    def withdraw(self, amount):
        return super().withdraw(amount + self.withdraw_fee)
```

super() = "the parent class's version, bound to self". Inside CheckingAccount.withdraw, super().withdraw(x) skips CheckingAccount and looks withdraw up starting in Account, so the **original** behavior is used. self is passed automatically.
- Same as Account.withdraw(self, x).
- **Not** self.withdraw(x); that finds CheckingAccount.withdraw which results in infinite recursion.
- Write self.withdraw_fee, not CheckingAccount.withdraw_fee, so a subclass or instance can override the fee.
- A subclass __init__ **overrides** the parent's; call super().__init__(...) to also run the parent's setup.

```
>>> ch = CheckingAccount('Tom')  # Account.__init__
>>> ch.interest     # Found in CheckingAccount
0.01
>>> ch.deposit(20)  # Found in Account
20
>>> ch.withdraw(5)  # Found in CheckingAccount
14
```

Looking up obj.name:
1. Evaluate obj to get an object.
2. If name is an **instance attribute** of that object, use it.
3. Otherwise look in the object's **class,** then its **base class,** then that class's base class, and so on.
4. Not found anywhere: AttributeError. If the value found is a function, you get a **bound method.**

Assignment (obj.name = v) never looks up: it sets name on obj itself.

Attribute or method? Same lookup. a.balance is a **value;** a.deposit is a **method** from the class that **runs only with ().** a.balance() errors: an int isn't callable.

Interfaces: objects of different classes that share attribute names (deposit, withdraw, balance) can be used the same way.